

Universidad Politécnica de Victoria

Ingeniería en Tecnologías de la Información

---

# Análisis Técnico de la Máquina de Estados Finitos (FSM) de la Máquina Expendedora

---

**Materia:** Fundamentos de Inteligencia Artificial

**Plataforma:** Arduino Mega 2560 (firmware C++, PlatformIO)

Arturo Zapata · Fabián Ramírez · Equipo

1 de octubre de 2026

# Índice

|                                                     |           |
|-----------------------------------------------------|-----------|
| <b>1. Introducción y Contexto del Proyecto</b>      | <b>2</b>  |
| <b>2. Punto de Entrada: main.cpp</b>                | <b>2</b>  |
| 2.1. Arquitectura de Super-Loop . . . . .           | 2         |
| 2.2. Inyección de Dependencias . . . . .            | 2         |
| <b>3. La Máquina de Estados: VmFsm</b>              | <b>3</b>  |
| 3.1. Concepto y Rol . . . . .                       | 3         |
| 3.2. Mecanismo de Transición . . . . .              | 3         |
| 3.3. Temporización No Bloqueante . . . . .          | 3         |
| 3.4. Interrelación con las Demás Clases . . . . .   | 4         |
| <b>4. Cálculo de Cambio: VmChangeCalculator</b>     | <b>4</b>  |
| 4.1. Denominaciones Aceptadas . . . . .             | 4         |
| 4.2. Algoritmo Greedy Decreciente . . . . .         | 4         |
| 4.3. Visualización del Cambio en Carrusel . . . . . | 5         |
| 4.4. Decisiones de Diseño . . . . .                 | 5         |
| <b>5. Los 13 Estados de la FSM</b>                  | <b>6</b>  |
| 5.1. Tabla Resumen . . . . .                        | 6         |
| 5.2. Descripción Detallada de Cada Estado . . . . . | 6         |
| <b>6. Diagrama de Maquina de Estados</b>            | <b>8</b>  |
| <b>7. Conclusiones</b>                              | <b>10</b> |

# 1. Introducción y Contexto del Proyecto

La máquina expendedora es un sistema embebido autónomo implementado en un **Arduino Mega 2560**. Todo el firmware corre en un único microcontrolador sobre un *super-loop* no bloqueante.

Los periféricos que gestiona el sistema son:

- **Teclado matricial 4×4** (D22–D29) — entrada del usuario.
- **LCD 20×4 I<sup>2</sup>C** (0x27) — retroalimentación visual.
- **Motores DC** vía PCA9685 (I<sup>2</sup>C) — despacho de producto.
- **Lector RFID MFRC522** (SPI: D53, D8) — pago con tarjeta.
- **Barrera óptica de caída** (D2, pull-up) — confirmación de entrega del motor.
- **EEPROM interna** — persistencia de inventario y caja.

El núcleo del firmware es el módulo `VmFsm`, una Máquina de Estados Finitos (FSM) con 13 estados que encapsula toda la lógica de negocio: selección de producto, cobro, despacho y cálculo de cambio.

## 2. Punto de Entrada: `main.cpp`

### 2.1. Arquitectura de Super-Loop

El firmware sigue el patrón *bare-metal super-loop*: `setup()` inicializa los periféricos una sola vez y `loop()` se ejecuta continuamente sin usar interrupciones ni RTOS.

```
1 void setup() {
2     Wire.begin(); // I2C antes del LCD y PCA9685
3     pinMode(VM_PIN_BARRIER, INPUT_PULLUP);
4     display.begin();
5     motor.begin();
6     rfid.begin();
7     fsm.begin(); // carga EEPROM y entra al estado
8     S0
9 }
10 void loop() {
11     char key = keypad.getKey(); // sondeo no bloqueante del
12     teclado
13     if (key != NO_KEY)
14         fsm.handleKey(key); // despachar tecla a la FSM
15     fsm.update(); // avanzar la maquina de estados
16 }
```

Listing 1: Núcleo del super-loop en `main.cpp`

### 2.2. Inyección de Dependencias

Todos los subsistemas se instancian como objetos estáticos globales y se *inyectan* al constructor de `VmFsm`:

```

1 static VmDisplay      display;
2 static VmEepromData   storage;
3 static VmMotorController motor;
4 static VmRfid          rfid;
5
6 static VmFsm fsm(storage, motor, rfid, onDisplay);

```

Listing 2: Instanciación e inyección en main.cpp

La función `onDisplay()` actúa como *callback* de presentación: la FSM nunca accede directamente al LCD; simplemente llama al puntero de función `DisplayFn` que le fue entregado en construcción.

## 3. La Máquina de Estados: VmFsm

### 3.1. Concepto y Rol

VmFsm implementa el patrón **Mediador/Controlador**: centraliza la lógica de negocio y coordina a los demás módulos sin que éstos se conozcan entre sí. Es una **Máquina de Moore**: las salidas (pantallas, motor, cambio) dependen únicamente del estado activo, y las transiciones dependen de eventos (teclas, RFID, temporizadores, barrera óptica).

### 3.2. Mecanismo de Transición

1. `enterState(FsmState next)` — cambia `_state` y llama al gancho `onEnter*()` correspondiente.
2. `update()` — llamado cada ciclo desde `loop()`. Sondea temporizadores, RFID y la barrera óptica para generar transiciones autónomas.
3. `handleKey()` — traduce el carácter físico a una acción semántica y la delega al procesador del estado activo.

### 3.3. Temporización No Bloqueante

Todos los temporizadores se implementan comparando `millis()` contra una referencia guardada, sin usar `delay()`:

```

1 _motorTimer = millis();                                     // arrancar
2
3 // En update():
4 if ((uint32_t)(millis() - _motorTimer) >= VM_TIMEOUT_MOTOR_MS)
5     // ... acción de timeout

```

Listing 3: Patrón de temporizador no bloqueante

| Constante                | Valor      | Uso                                   |
|--------------------------|------------|---------------------------------------|
| VM_TIMEOUT_INACTIVITY_MS | 180 000 ms | Regreso a reposo en S3–S6             |
| VM_TIMEOUT_MOTOR_MS      | 10 000 ms  | Tiempo máximo de giro del motor en S8 |
| VM_CAROUSEL_INTERVAL_MS  | 2 000 ms   | Rotación de subpantalla del carrusel  |

Cuadro 1: Temporizadores de la FSM

### 3.4. Interrelación con las Demás Clases

| Clase              | Responsabilidad                                                                 |
|--------------------|---------------------------------------------------------------------------------|
| VmEepromData       | Persistencia de slots (nombre, precio, stock) y caja de efectivo.               |
| VmMotorController  | Controla motores DC vía PCA9685; detecta entrega mediante barrera óptica (D2).  |
| VmRfid             | Lectura de tarjetas MIFARE y escritura del saldo tras el cobro.                 |
| VmKeypad           | Interpreta caracteres del teclado 4×4 en acciones semánticas según el contexto. |
| VmDisplay          | Abstracción del LCD 20×4 I <sup>2</sup> C; recibe 4 cadenas de texto.           |
| VmCarousel         | Rota subpantallas cada 2 s llamando al callback de display.                     |
| VmChangeCalculator | Algoritmo voraz de cálculo de cambio sobre las monedas de la caja.              |

Cuadro 2: Módulos del firmware y sus responsabilidades

## 4. Cálculo de Cambio: VmChangeCalculator

### 4.1. Denominaciones Aceptadas

El sistema maneja cuatro denominaciones de moneda mexicana (máximo \$10):

\$1.00 (100c)    \$2.00 (200c)    \$5.00 (500c)    \$10.00 (1000c)

En el teclado, la correspondencia es: 1=\$1, 2=\$2, 3=\$5, 4=\$10. Las teclas 5–9 se ignoran.

### 4.2. Algoritmo Greedy Decreciente

El cambio se calcula con un **algoritmo voraz**: se intentan primero las monedas de mayor denominación, reduciendo el monto restante hasta completarlo o agotar existencias.

```

1  const uint32_t denominaciones[4] = { 1000u, 500u, 200u, 100u };
2
3  for (uint8_t i = 0; i < 4 && total > 0u; i++) {
4      uint32_t disponible = 0;
5      cashBox.getCoinStock(denominaciones[i], disponible);
6
7      uint32_t usar = 0;
8      while (usar < disponible && total >= denominaciones[i]) {
9          total -= denominaciones[i];
10         usar++;
11     }
12     if (usar > 0u) {
13         cashBox.deductCoins(denominaciones[i], usar);
14         *contadores[i] = usar;
15     }
16 }
17 if (total > 0u) return false; // cambio no realizable

```

Listing 4: Núcleo del algoritmo greedy en VmChangeCalculator::calculate()

### 4.3. Visualización del Cambio en Carrusel

Tras un pago exitoso en efectivo, la FSM construye un **carrusel de slides**: un slide inicial de éxito seguido de uno por cada denominación no nula (de mayor a menor). Las denominaciones con cero monedas se omiten:

```

1  _carousel.addScreen(finSuccessTrampoline); // slide 0: Compra
    Exitosa
2
3  const uint32_t denoms[4] = {1000u, 500u, 200u, 100u};
4  for (uint8_t i = 0; i < 4; i++) {
5      if (qtys[i] > 0u) {
6          _changeDenoms[_changeSlides] = denoms[i];
7          _changeQtys[_changeSlides] = qtys[i];
8          _changeSlides++;
9          _carousel.addScreen(changeScreenTrampoline);
10     }
11 }

```

Listing 5: Construcción dinámica del carrusel de cambio en onEnterPantallaFin()

Ejemplo: si el cambio es \$7.00, el carrusel muestra tres slides: *Compra Exitosa* → *\$5.00 × 1 pieza* → *\$2.00 × 1 pieza*.

### 4.4. Decisiones de Diseño

- **Aritmética entera en centavos**: sin errores de coma flotante en AVR de 8 bits.
- **Descuento inmediato en EEPROM**: cada deductCoins() persiste al instante.
- **Retorno false si no hay fondos**: mensaje 5 s y regreso a reposo.

## 5. Los 13 Estados de la FSM

### 5.1. Tabla Resumen

| ID       | Nombre             | Propósito                                                   | Transiciones de salida                                                      |
|----------|--------------------|-------------------------------------------------------------|-----------------------------------------------------------------------------|
| $S_0$    | S0_START           | Inicialización; muestra mensaje de bienvenida.              | $\rightarrow S_2$ inmediatamente.                                           |
| $S_1$    | S1_INTERNAL_ERROR  | Error crítico de EE-PROM; sistema detenido.                 | Estado terminal.                                                            |
| $S_2$    | S2_STANDBY         | Carrusel del catálogo (2 slides de 2 productos).            | $\rightarrow S_3$ (tecla 1–4).                                              |
| $S_3$    | S3_SELECT_CHANNEL  | Confirmar producto elegido.                                 | $\rightarrow S_4$ [A] / $S_2$ [*] / timeout.                                |
| $S_4$    | S4_SELECT_PAYMENT  | Elegir método: efectivo o RFID.                             | $\rightarrow S_5$ [A] / $S_6$ [B] / $S_2$ [*].                              |
| $S_5$    | S5_WAIT_CASH       | Acumular monedas (teclas 1–4: \$1/\$2/\$5/\$10).            | $\rightarrow S_7$ (monto $\geq$ precio) / $S_2$ [B] / timeout.              |
| $S_6$    | S6_WAIT_RFID       | Sondear RFID y cobrar saldo.                                | $\rightarrow S_7$ (OK) / $S_2$ (error/timeout/[B]).                         |
| $S_7$    | S7_PREPARING_VEND  | Reservar stock y arrancar motor.                            | $\rightarrow S_8$ (OK) / $S_{10}$ (motor falló) / $S_2$ (sin stock, 1.5 s). |
| $S_8$    | S8_DISPENSING      | Motor activo; esperar barrera óptica.                       | $\rightarrow S_9$ (barrera OK) / $S_{10}$ (timeout 10 s).                   |
| $S_9$    | S9_CONFIRMED       | Entrega confirmada; esperar 1 s.                            | $\rightarrow S_{11}$ (efectivo) / $S_{12}$ (RFID).                          |
| $S_{10}$ | S10_VEND_ERROR     | Rollback de stock; mensaje de error 3 s.                    | $\rightarrow S_2$ .                                                         |
| $S_{11}$ | S11_CALC_CHANGE    | Algoritmo greedy; preparar slides de cambio.                | $\rightarrow S_{12}$ (éxito) / $S_2$ (sin fondos, 5 s).                     |
| $S_{12}$ | S12_FINISH_SCREEN  | Carrusel: éxito + una diapositiva por denominación no nula. | $\rightarrow S_2$ .                                                         |
| $S_{13}$ | S13_MESSAGE_PROMPT | Estado transitorio no bloqueante para mensajes (espera A).  | Depende.                                                                    |

Cuadro 3: Los 13 estados de la FSM de la máquina expendedora

### 5.2. Descripción Detallada de Cada Estado

$S_0$  — **Arranque.** Muestra mensaje de bienvenida; en el primer ciclo de `update()` transita a  $S_2$ .

$S_1$  — **Falla Interna.** Estado terminal por error de EEPROM. Requiere reinicio físico del sistema.

$S_2$  — **Reposo.** Carrusel con dos slides: el primero muestra los productos 1 y 2; el segundo, los productos 3 y 4 (cada 2 s).

$S_3$  — **Selección de Canal.** Consulta `getSlot()` para verificar que el slot está habilitado. Muestra nombre y precio del producto.

$S_4$  — **Selección de Pago.** El usuario elige la pasarela de cobro. Si RFID no está disponible, vuelve a esta pantalla con un aviso.

$S_5$  — **Espera de Efectivo.** Teclas 1–4 representan monedas de \$1, \$2, \$5 y \$10. Cada pulsación llama a `addCoins()` y actualiza el acumulado. Al cubrir el precio transita automáticamente a  $S_7$ . La pantalla muestra lo insertado, lo que falta y las teclas válidas.

$S_6$  — **Espera de RFID.** `VmRfid::poll()` se llama en cada ciclo sin bloquear. Saldo suficiente  $\rightarrow$  `deductBalance()`  $\rightarrow S_7$ . Saldo insuficiente o error de lectura  $\rightarrow$  mensaje y  $S_2$ .

$S_7$  — **Reservada.** Punto de no retorno transaccional: (1) `reserveStock()` decrementa el inventario en EEPROM; (2) `motor.start()` verifica que la barrera óptica esté libre y enciende el motor. Si `start()` falla, transita a  $S_{10}$ .

$S_8$  — **Dispensando.** Supervisa `motor.isBusy()` en cada ciclo. La barrera óptica al detectar la caída del producto detiene el motor y reporta `VM_RESULT_DELIVERED`. Un timeout de 10 s protege contra atascos.

$S_9$  — **Confirmada.** Muestra pantalla de éxito 1 s. Luego, si el pago fue en efectivo, avanza a  $S_{11}$ ; si fue RFID (sin cambio), a  $S_{12}$ .

$S_{10}$  — **Falla de Despacho.** Si `_stockReserved` es `true`, llama a `releaseStock()` para devolver la unidad al inventario. Muestra mensaje de error 3 s y retorna a  $S_2$ .

$S_{11}$  — **Cálculo de Cambio.** Invoca `VmChangeCalculator::calculate()`. En caso de éxito, construye los arrays `_changeDenoms[]` y `_changeQtys[]` para el carrusel. Si no hay fondos suficientes, muestra aviso y espera 5 s antes de volver a  $S_2$ .

$S_{12}$  — **Pantalla Final.** El carrusel muestra: slide de éxito, seguido de un slide por cada denominación no nula (de \$10 a \$1) o el resumen de cobro RFID. Regresa a  $S_2$  al completar todos los slides + 1 s.

$S_{13}$  — **Prompt de Mensaje.** Estado transitorio no bloqueante para mostrar mensajes al usuario y esperar la confirmación. Transita al estado destino configurado tras pulsar la tecla A.

## 6. Diagrama de Maquina de Estados

Cada estado muestra sus condiciones de entrada y de salida con el estado destino. La Figura 1 presenta el diagrama completo en la siguiente pagina.

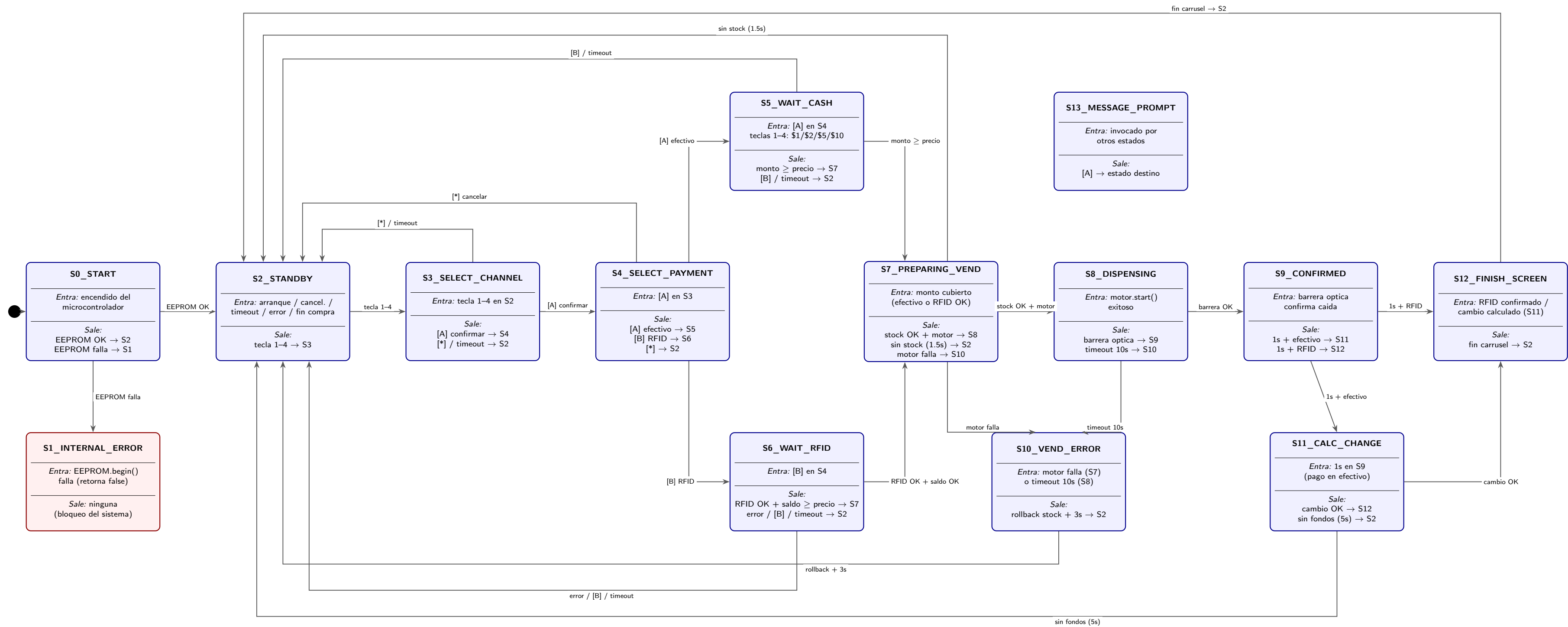


Figura 1: Diagrama de estados UML de la FSM de la máquina expendedora (13 estados). Cada estado muestra sus condiciones de entrada (*Entra:*) y de salida (*Sale:*) con el estado destino. Todas las transiciones siguen trayectorias ortogonales.

## 7. Conclusiones

El diseño de la FSM cumple los objetivos de un sistema embebido robusto y legible:

1. **Legibilidad lineal:** los 13 estados están numerados  $S_0$ – $S_{12}$  y el código fuente sigue el mismo orden de arriba a abajo, facilitando la lectura y el mantenimiento.
2. **Sin sobreingeniería:** cada clase tiene una responsabilidad única. Se eliminaron el registro transaccional (EEPROM) y el sensor de puerta, que no corresponden al hardware real actual.
3. **Visualización correcta del cambio:** el carrusel de  $S_{12}$  muestra dinámicamente solo las denominaciones no nulas, de mayor a menor, una por slide, facilitando la lectura al usuario.
4. **Denominaciones reales:** el sistema acepta exclusivamente monedas de \$1, \$2, \$5 y \$10 (teclas 1–4), coherente con el hardware de la caja real.
5. **Temporización no bloqueante:** todos los delays se implementan con `millis()`, permitiendo que el super-loop permanezca siempre receptivo a eventos externos.
6. **Carrusel corregido:** el módulo `VmCarousel` incrementa correctamente el índice en cada ciclo y muestra la primera slide de forma inmediata al entrar a un nuevo estado.